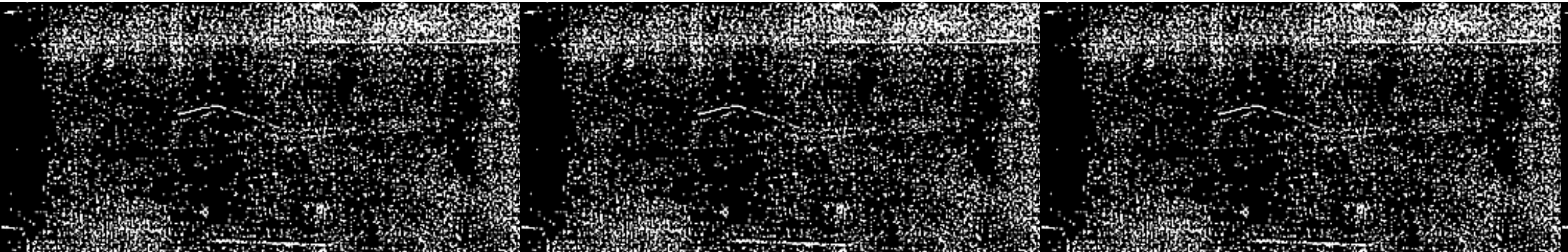
WEB PROGRAMMING

Benoit Gaudou
benoit.gaudou@ut-capitole.fr

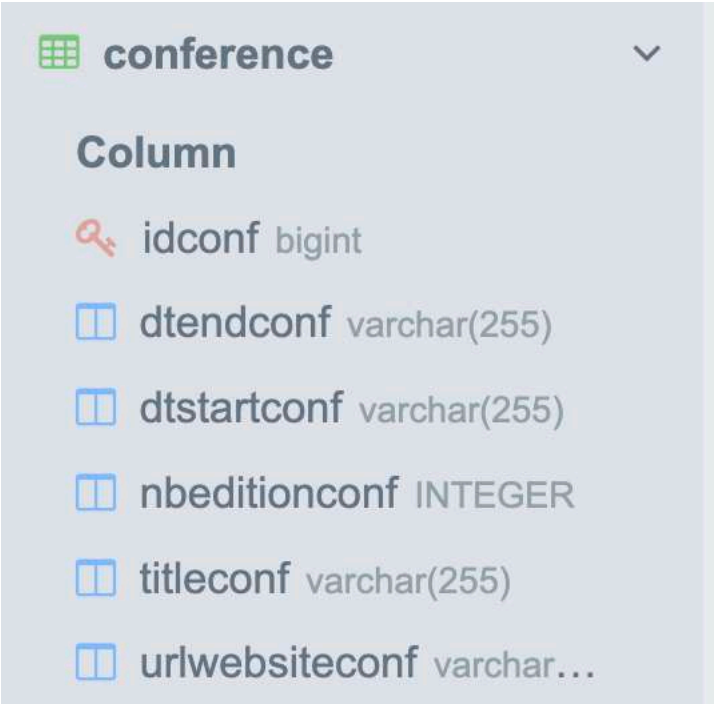
UNIVERSITÉ TOULOUSE 1 CAPITOLE

GESTION DES DONNÉES



OBJECT-RELATIONAL MAPPING (ORM)

- Conceptuellement :
  - Une **Table** est proche d'une **Class**
  - Une **ligne d'une Table** est proche d'un **Object**




```
public class Conference {

        private Long idconf;

        private String titleconf;
        private Integer nbeditionconf;
        private String dtstartconf;
        private String dtendconf;
        private String urlwebsiteconf;
```

OBJECT-RELATIONAL MAPPING (ORM)

- Un mapping Objet-Relationnel ou **mapping O/R** (**Object-Relational Mapping, ORM)** fait correspondre objets Java à des tables de base de données relationnelle.

- **Pourquoi utiliser un ORM?**
  - **Cela simplifie l'accès à la BD** (Moins besoin d'écrire du code SQL à la main).
  - **Automatise** la conversion Object ⇄ Table.
  - Améliore la **lisibilité et la maintenabilité** du code (code principalement orienté objet).
  - **Indépendance du SGBD** – Fonctionne avec MySQL, PostgreSQL, etc., sans modifier les requêtes.
  - Gère les problèmes de **conversion de type**

OBJECT-RELATIONAL MAPPING (ORM)

- Java a introduit l'API **JPA (Java Persistence API)** qui fournit un mécanisme de gestion de la persistance et du mappage objet-relationnel.

- **Framework ORM en Java**
  - Hibernate – le plus couramment utilisé, implémente JPA.
  - Spring Data JPA – simplifie l'utilisation de Hibernate et JPA.

SGBD (CONFIGURATION)

- JPA permet du code independent du SGBD
  - Seulement besoin de configurer l'accès au serveur de BD ==> dans le fichier application.properties

```
# Configuration of the DBMS SQLite
spring.datasource.url=jdbc:sqlite:db/conferences.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.datasource.username=
spring.datasource.password=

# Configuration of Hibernate
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

```
# Configuration of the DBMS
spring.datasource.url=jdbc:mysql://localhost:3306/my_base
spring.datasource.username=root
spring.datasource.password=password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Configuration of JPA and Hibernate
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

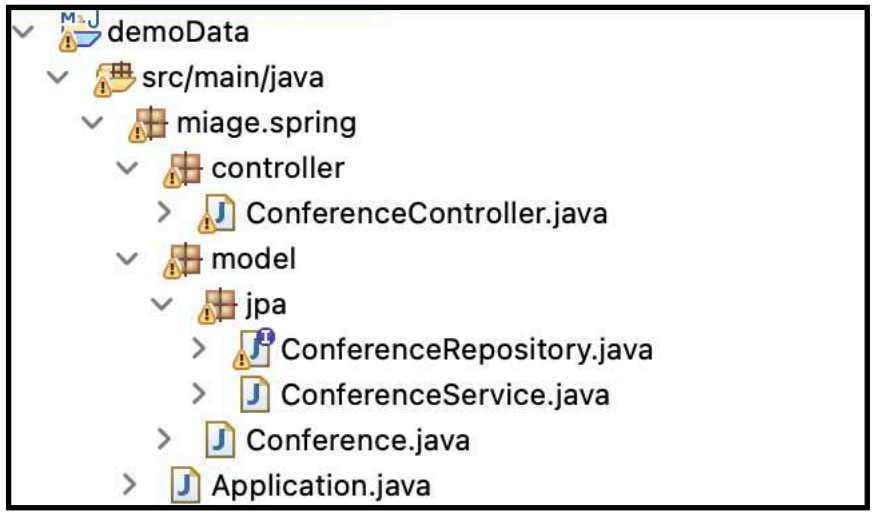
ARCHITECTURE

- Pour chaque table, on **créera une Classe Java** avec des **annotations** spécifiques
  p.e. Conference

- **Création d'un Repository (Interface)**
  Cela donne accès à des opérations pré-définies sur la base de données ou de développer ses propres requêtes ==> **SQL sera limité à ce fichier !**

- **Création d'un Service**
  il utilise le Repository crée pour gérer la partie logique de l'application

- **Utilisation du Service dans le Controller**



La Création d'instances est faite par **Injection de Dépendence** en utilisant l'annotation **@Autowired**

ORM: CRÉER LA CLASSE

- **Exemple:** on veut créer un site web qui affiche/gère les utilisateurs qui veulent s'inscrire à une ou plusieurs conférences.

- **Création de la classe User**

```
public class User {

   private Long id;

    private String name;
    private String email;

   // + setter / getter
}
```

ORM: CRÉER LA CLASSE

- **Exemple:** on veut créer un site web qui affiche/gère les utilisateurs qui veulent s'inscrire à une ou plusieurs conférences.

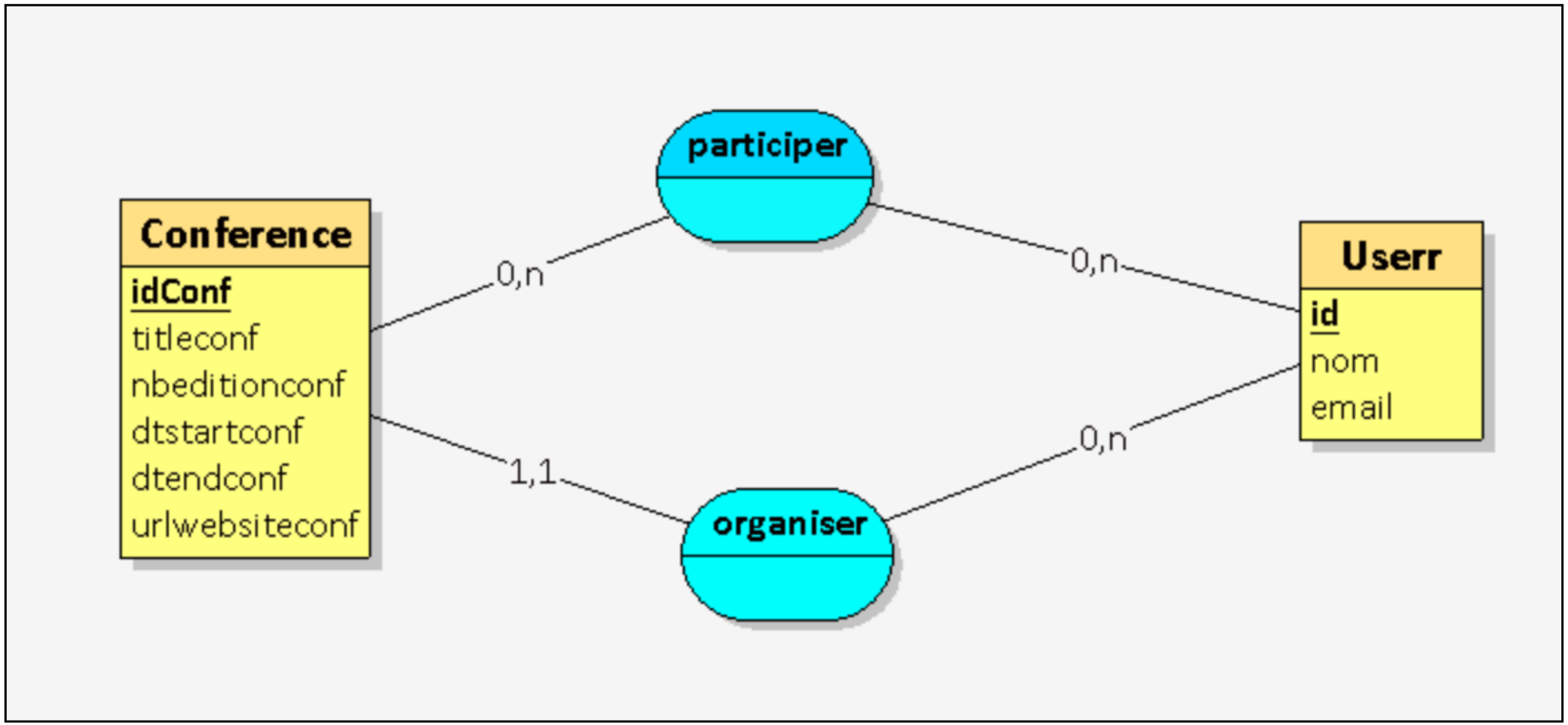
- **Création de la classe User**

```
public class User {

   private Long id;

    private String name;
    private String email;

   // + setter / getter
}
```

- **Ajouts d'annotations**

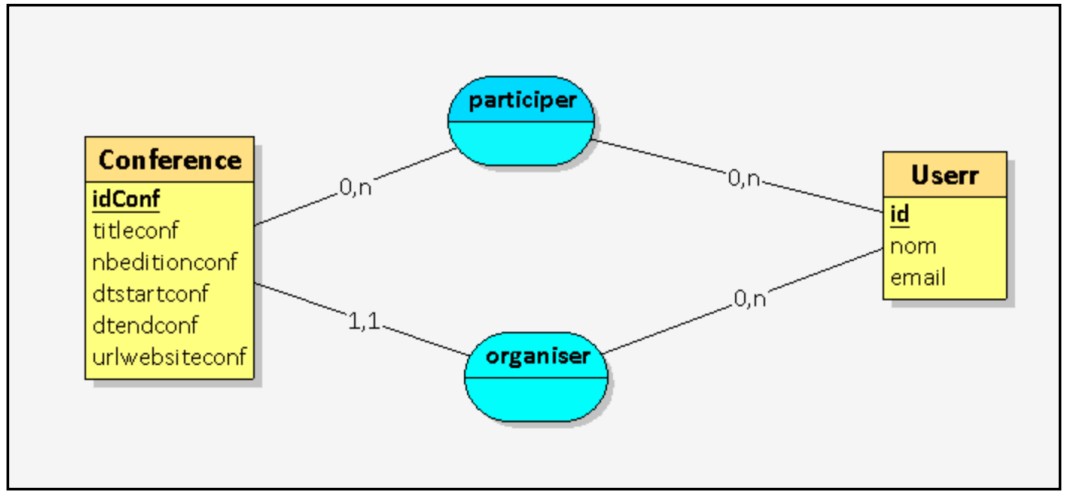
```
import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // + setter / getter
}
```

LIENS ENTRE ENTITÉS



LIENS ENTRE ENTITÉS : LES 2 CLASSES

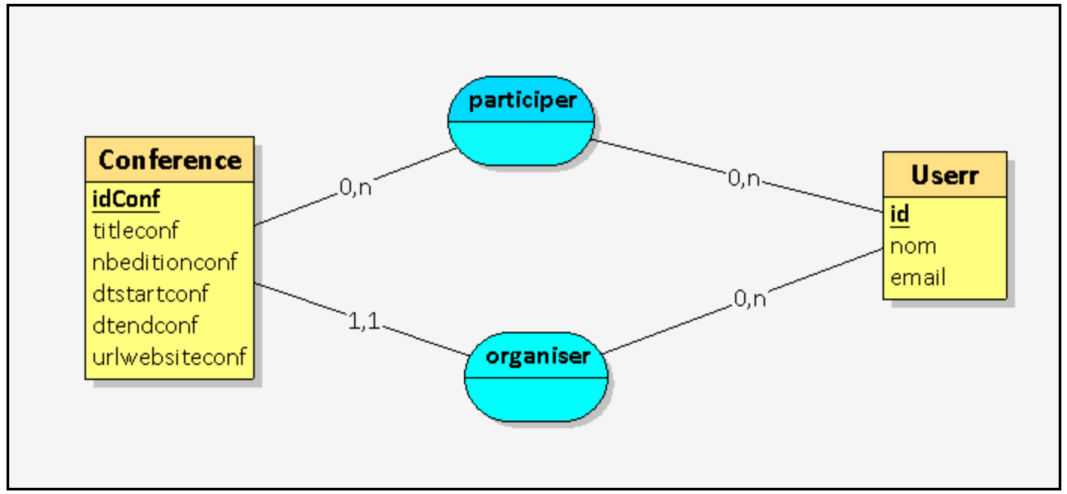
```java
@Entity
@Table(name = "conference")
public class Conference {

     @Id
     @GeneratedValue(strategy = GenerationType.AUTO)
//    @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long idconf;

     private String titleconf;
     private Integer nbeditionconf;
     private String dtstartconf;
     private String dtendconf;
     private String urlwebsiteconf;
```




```java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;
    private String email;
```

LIENS ENTRE ENTITÉS (PUREMENT OBJET)

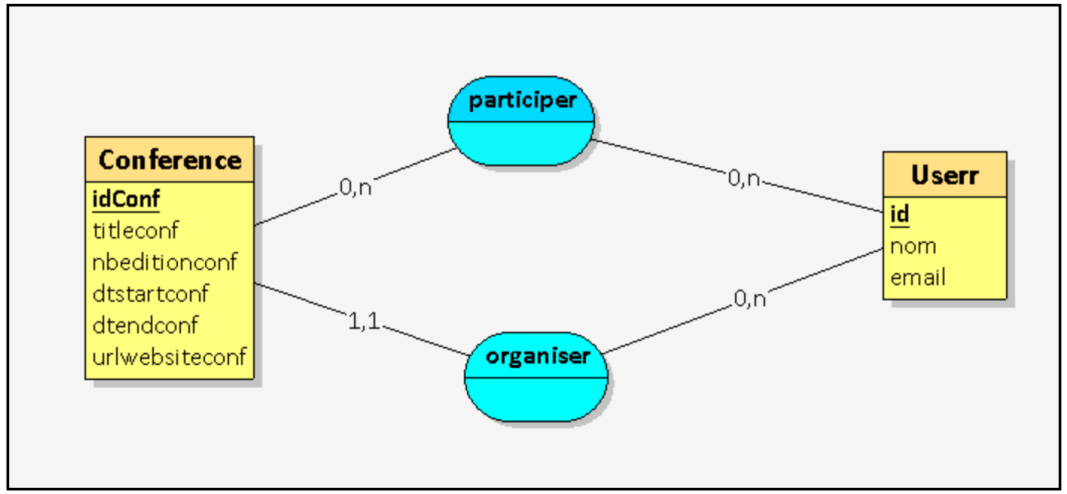
```java
@Entity
@Table(name = "conference")
public class Conference {

     @Id
     @GeneratedValue(strategy = GenerationType.AUTO)
//    @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long idconf;

     private String titleconf;
     private Integer nbeditionconf;
     private String dtstartconf;
     private String dtendconf;
     private String urlwebsiteconf;


     // 0 à n User peuvent participer à une conférence
     private ArrayList<User> participants = new ArrayList<>();

     // 1 (et 1 seul) User organise une conférence
     private User organizer;
```




```java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;
    private String email;
```

LIENS ENTRE ENTITÉS

```java
@Entity
@Table(name = "conference")
public class Conference {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
//    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idconf;

    private String titleconf;
    private Integer nbeditionconf;
    private String dtstartconf;
    private String dtendconf;
    private String urlwebsiteconf;

    @ManyToMany                                    // new table
    @JoinTable(
        name = "conference_participants",
        joinColumns = @JoinColumn(name = "idconf"),
        inverseJoinColumns = @JoinColumn(name = "id")
    )
    private ArrayList<User> participants = new ArrayList<>();

    @ManyToOne
    @JoinColumn(name = "organizer_id")// Foreign key in Conference table
    private User organizer;
```




```java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;
    private String email;
```

LIENS ENTRE ENTITÉS

```
@ManyToMany                // new table
@JoinTable(
    name = "conference_participants",
    joinColumns = @JoinColumn(name = "idconf"),
    inverseJoinColumns = @JoinColumn(name = "id")
)
private ArrayList<User> participants = new ArrayList<>();
```

```
1 SELECT * FROM conference_participants
```

idconf	id

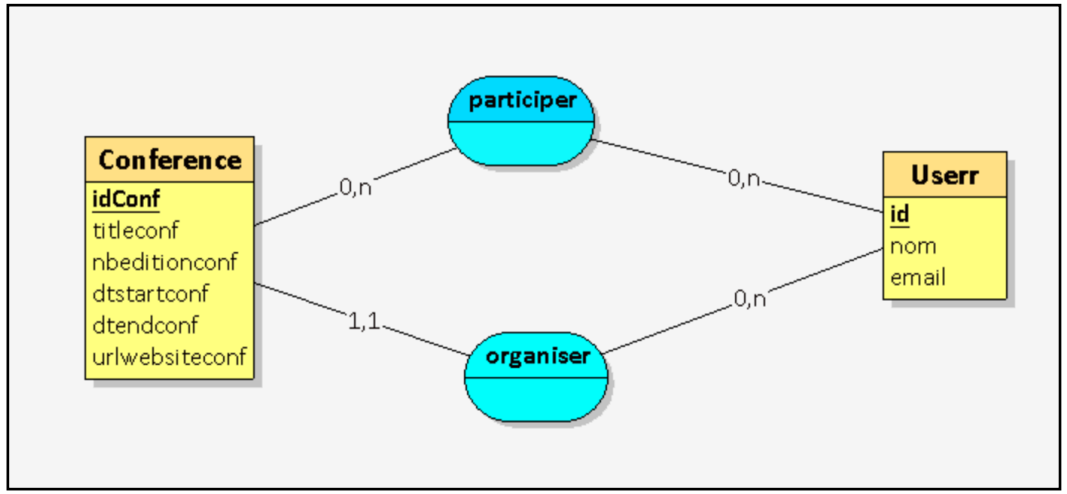
LIENS ENTRE ENTITÉS

```
@ManyToOne
@JoinColumn(name = "organizer_id")// Foreign key in Conference table
private User organizer;
```

```
1 SELECT * FROM conference
```

idc…	dtend…	dtstar…	nbedit…	partici…	titleconf	urlwe…	organizer_id
1	2025-0…	2025-0…	1	BLOB	MIAGE	ss.fr	NULL

LIEN ENTRE ENTITÉS : CODE SQL GÉNÉRÉ




```
Hibernate: create table HTE_conference(nbeditionconf integer, rn_ integer not null, idconf bigint, organizer_id
bigint, hib_sess_id char(36) not null, dtendconf varchar(255), dtstartconf varchar(255), titleconf varchar(255),
urlwebsiteconf varchar(255), primary key (rn_, hib_sess_id))

Hibernate: create table conference (idconf bigint not null, dtendconf varchar(255), dtstartconf varchar(255),
nbeditionconf integer, titleconf varchar(255), urlwebsiteconf varchar(255), organizer_id bigint, primary key
(idconf))
Hibernate: create table conference_participants (idconf bigint not null, id bigint not null)
Hibernate: create table conference_seq (next_val bigint)
Hibernate: insert into conference_seq values ( 1 )

Hibernate: create table users (id integer, email varchar(255), nom varchar(255), primary key (id))
```

REPOSITORY

- En JPA, un Repository est une **interface** fournissant des requêtes d'accès sans coder de SQL.
  Il infére la requête à partir du nom de la méthode : e.x. `findBy + Attribute name`

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
       List<User> findByEmail(String email);
      List<User> findByEmailAndName(String email, String name);
    }
```

- Il est possible de rajouter nos propres requêtes en utilisant SQL.

```
public interface ConferenceRepository extends JpaRepository<Conference, Long>  {

    List<Conference> findByTitleconf(String titleconf);

    @Query("SELECT c FROM Conference c WHERE c.nbeditionconf = 1")
    List<Conference> findWithNumOne();


    @Query("SELECT c FROM Conference c WHERE LOWER(c.titleconf) = LOWER(:titleconf)")
    Conference findConfTitle(@Param("titleconf") String titleconf);
}
```

findByTitleconf se base sur l'attribut nommé **titleConf**

SERVICE

- A partir d'une instance de UserRepository, obtenue par Injection de Dependence, UserService implémente certains des services sur la BD.

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepo;

    public List<User> getAllUsers() {
        return userRepo.findAll();
    }
    public User createUser(User u) {
        return userRepo.save(u);
    }

    public User findUserById(Long id) {
        return userRepo.findUserById(id);
    }

}
```

REPOSITORY

- En JPA, un Repository est une **interface** fournissant des requêtes d'accès sans coder de SQL.

```
23          return userRepo.findUserById(id);
24      }
25
26      public void test() {
27          userRepo.
28      }
29
30  }
31
```

findAll(Sort sort) : List<User> - ListPagingAndSortingRepos
findAll(Example<S> example, Pageable pageable) : Page<S
findAll(Example<S> example, Sort sort) : List<S> - JpaRepo
findAllById(Iterable<Long> ids) : List<User> - ListCrudRepo
findBy(Example<S> example, Function<FetchableFluentQu
findById(Long id) : Optional<User> - CrudRepository
findOne(Example<S> example) : Optional<S> - QueryByExa
findUserById(Long id) : User - UserRepository
flush() : void - JpaRepository
getById(Long id) : User - JpaRepository
getClass() : Class<?> - Object
Press '^Space' to show Template Proposals

Returns the number of
Returns:
the number of

EXEMPLE POUR LES USERS

```java
public interface UserRepository extends JpaRepository<User, Long>  {

   public User findById(Long id);

}
```

```java
@Service
public class UserService {
    @Autowired
    private UserRepository userRepo;

    public List<User> getAllUsers() {
        return userRepo.findAll();
    }
    public User createUser(User c) {
        return userRepo.save(c);
    }

    public User findUserById(Long id) {
        return userRepo.findById(id);
    }

}
```

CONTROLLER

- Ajout d'une variable de type UserService dans la classe Controller.
- Elle est instanciée par **Injection de Dependence**.

```java
@Controller
@RequestMapping("/users")
public class UserController {
    @Autowired
    private UserService userService;

    @GetMapping
    public String getAllUsers(Model model) {
        List<User> users = userService.getAllUsers();
        model.addAttribute("users",users);
        return "users";
     }

    @GetMapping("/{id}")
    public User getUserById(@PathVariable Long id) {
        return userService.findUserById(id);
    }


    @GetMapping("/insertUser")
    public String insertUser(
       @RequestParam(name="txt_nom", required=true) String name,
       @RequestParam(name="txt_email", required=true) String email
    ) {
       User u = new User();
       u.setNom(name);
       u.setEmail(email);

       userService.createUser(u);
       return "OK";
     }

}
```